

Current architecture JSON contains only:

- Components
- Ports
- Registers
- Connections

This is sufficient for basic graph visualization but insufficient for professional architecture diagrams.

To support advanced visualization, analysis, documentation, validation, code generation, and IP-XACT-style exploration, extend the schema with the following metadata.

1. Component Category

Add explicit categories.

Example:

```
{  
  "name": "UART0",  
  "type": "Peripheral",  
  "category": "UART"  
}
```

Possible categories:

- CPU
- Bus
- Memory
- DMA
- InterruptController
- Timer
- UART
- SPI
- I2C
- GPIO
- ADC
- DAC
- PWM
- Ethernet
- CAN
- Debug
- Clock
- Reset
- CustomIP

2. Hierarchical Groups

Allow components to belong to groups.

Example:

```
{  
  "name": "UART0",  
  "group": "Communication Peripherals"  
}
```

Possible groups:

- Processing
- Memory Subsystem
- Communication Peripherals
- Analog Peripherals
- Timing Peripherals
- Interrupt Subsystem
- Debug Subsystem

3. Port Direction

Current ports are only names.

Instead:

```
{  
  "name": "tx",  
  "direction": "output"  
}
```

or

```
{  
  "name": "rx",  
  "direction": "input"  
}
```

Possible values:

- input
- output
- bidirectional

4. Port Type

Specify signal meaning.

Example:

```
{  
  "name": "irq",  
  "direction": "output",  
  "signalType": "interrupt"  
}
```

Possible values:

- bus
- interrupt
- clock
- reset
- data
- control
- debug
- dma
- analog

5. Port-Level Connections

Instead of:

```
{
```

```
"source": "UART0",  
"target": "INTC"  
}
```

Use:

```
{  
  "source": {  
    "component": "UART0",  
    "port": "irq"  
  },  
  "target": {  
    "component": "INTC",  
    "port": "irq_in"  
  }  
}
```

This enables true multi-handle routing.

6. Connection Type

Add semantic meaning.

```
{
```

```
"source": "...",  
"target": "...",  
"type": "interrupt"  
}
```

Possible values:

- bus
- interrupt
- clock
- reset
- dma
- control
- debug
- analog

7. Bus Metadata

For buses:

```
{  
  "name": "AHB_BUS",  
  "busType": "AHB",  
  "width": 32,
```

```
"frequencyMHz": 100
}
```

Supported examples:

- AXI
- AHB
- APB
- Wishbone
- Avalon
- PCIe

8. Memory Metadata

```
{
  "name": "FLASH",
  "memoryType": "Flash",
  "sizeKB": 512,
  "width": 32
}
```

9. Register Metadata

Instead of register names only:

```
{  
  "name": "CTRL",  
  "offset": "0x00",  
  "size": 32,  
  "access": "RW"  
}
```

Enables register viewers.

10. Physical Coordinates (Optional)

Allow manual placement override.

```
{  
  "layout": {  
    "x": 100,  
    "y": 200  
  }  
}
```

Useful for custom architecture diagrams.

11. Clock Domains

```
{  
  "clockDomain": "SYS_CLK"  
}
```

Used for:

- clock tree visualization
- CDC analysis

12. Reset Domains

```
{  
  "resetDomain": "SYS_RST"  
}
```

Used for reset analysis.

13. Interrupt Metadata

```
{
  "interrupts": [
    {
      "name": "uart_irq",
      "vector": 12,
      "priority": 2
    }
  ]
}
```

14. Documentation Support

```
{
  "description": "UART peripheral supporting full-duplex serial communication."
}
```

Used for tooltips and architecture documentation.

15. IP-XACT Compatibility

Keep fields mappable to:

- vendor
- library
- name
- version (VLNV)

Example:

```
{  
  "vendor": "Acme",  
  "library": "PeripheralLib",  
  "name": "UART",  
  "version": "1.0"  
}
```

This allows future import/export to standard IP-XACT formats.

Goal:

The JSON should become a complete architecture description capable of supporting:

- React Flow visualization
- ELK layout optimization
- Architecture exploration
- Register inspection
- Documentation generation

- IP-XACT export/import
- RTL generation assistance
- Hardware dependency analysis
- Interrupt analysis
- Clock-domain analysis
- Bus topology analysis